

04 — STL CONTAINERS, ITERATORS, ALGORITHMS & PERFORMANCE

Interviewers expect you to pick the right container by complexity + memory layout, and to know the invalidation rules cold.

How to think about this. Choosing a container is answering two questions, in order: (1) *what's my access pattern* — random index, both ends, ordered traversal, key lookup? That narrows you to a couple of candidates by big-O. (2) *what's the memory layout* — contiguous (vector/array) or pointer-chasing (list/map/node)? On real hardware the second question often overrules the first, because a cache miss costs ~100× an L1 hit. Start from "vector until proven otherwise" and make the container justify a *departure* from contiguity.

Take-home lesson. Big-O tells you how cost *scales*; cache lines tell you what the cost *is*. For the n you actually have, a contiguous `vector` with a "worse" complexity usually beats a node-based container with a "better" one. Pick the data structure that matches your access pattern *and* the hardware.

1. CONTAINER COMPLEXITY & LAYOUT (THE CHEAT SHEET)

Container	Access	Insert/erase	Memory / notes
<code>vector</code>	$O(1)$ random	$O(1)$ amortized push_back; $O(n)$ middle	contiguous; cache-friendly; the default
<code>array</code>	$O(1)$	fixed size	stack, no heap, zero overhead
<code>deque</code>	$O(1)$ random	$O(1)$ both ends; $O(n)$ middle	chunked; not contiguous
<code>list</code>	$O(n)$	$O(1)$ anywhere (given iterator)	doubly-linked; poor locality; rarely worth it
<code>forward_list</code>	$O(n)$	$O(1)$	singly-linked; minimal footprint
<code>map</code> / <code>set</code>	$O(\log n)$	$O(\log n)$	red-black tree; ordered; node-based
<code>unordered_map</code> / <code>set</code>	$O(1)$ avg, $O(n)$ worst	$O(1)$ avg	hash table; buckets + chaining
<code>multimap</code> / <code>multiset</code>	$O(\log n)$	$O(\log n)$	duplicates allowed

Rules of thumb:

- Default to `vector`. Contiguity beats theoretical big-O for typical n because of cache lines and prefetching. A `list` with $O(1)$ insert usually loses to a `vector` with $O(n)$ insert until n is large.

War story (Stroustrup's own). In a famous talk, inserting/removing elements in sorted order was benchmarked: `vector` (shift everything, $O(n)$ per op) vs `list` (splice, $O(1)$ per op). Intuition says `list` wins. It loses — badly — at every size that fits real memory, because `vector`'s linear scan is a cache-friendly sequential sweep while `list` chases pointers all over the heap (cache miss per node). The $O(1)$ operation on the wrong memory layout is slower than the $O(n)$ operation on the right one.

- `deque` when you need fast push/pop at both ends and stable *references* (but not iterators) across end insertions.
- Ordered (`map`) vs hashed (`unordered_map`): choose ordered when you need ordering, range queries, or worst-case guarantees; hashed for raw lookup speed with a good hash.

2. ITERATOR CATEGORIES & INVALIDATION

Categories (weakest→strongest): input / output → forward → bidirectional → random-access → (C++20) contiguous. Algorithm requirements are stated in these terms (`std::sort` needs random-access; `std::list::sort` is a member because list iterators aren't).

Invalidation rules (memorize — very commonly asked):

- `vector`: any reallocation (growth past capacity) invalidates *all* iterators/pointers/refs. Insert/erase invalidates everything at/after the point. `reserve` avoids reallocation.
- `deque`: insert/erase in the middle invalidates all; at the ends invalidates iterators but not references.
- `list` / `forward_list`: only the erased element's iterators invalidate; everything else stable.
- `map` / `set` (node-based): iterators/refs stable except for erased elements.
- `unordered_*`: rehash invalidates iterators (not references/pointers to elements). Insert can trigger rehash.
- The erase-remove idiom (pre-C++20) / `std::erase` / `std::erase_if` (C++20+):

```
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end()); // classic
std::erase_if(v, pred);                                     // C++20
```

Stop and think. What's wrong with `for (auto it = v.begin(); it != v.end(); ++it) if (*it == x) v.erase(it);` on a `std::vector`?

► answer

3. `VECTOR` INTERNALS & PERF LEVERS

- Three pointers: begin, end (size), end-of-storage (capacity).
- Growth factor ~1.5–2× → amortized O(1) push_back.

- `reserve(n)` before a known-size fill to avoid repeated reallocation + moves. `shrink_to_fit` (non-binding request).
- `size()` vs `capacity()`. `clear()` keeps capacity; swap-with-empty frees it (pre-C++11) — now `shrink_to_fit`.

`clear()` empties, it doesn't deallocate. It destroys the elements and sets `size()` to 0 but keeps `capacity()` — the buffer is retained so you can refill without reallocating. That retention is a feature (cheap reuse), not a leak. If you genuinely need the memory back, call `shrink_to_fit()` (non-binding) or swap with a fresh empty vector. `std::string` behaves the same way.

- `emplace_back` vs `push_back`: `emplace` constructs in place from args (avoids a temporary); `push_back` takes an already-built object (can move). Not always faster; `emplace` can suppress useful conversions/ `explicit` checks.
 - `std::vector<bool>` is a space-optimized bitset, not a real container of `bool` — no `data()`, proxy references. Avoid in generic code.
-

4. HASH CONTAINERS INTERNALS

- Bucket array + chaining (typically). `load_factor = size/bucket_count`; rehash when it exceeds `max_load_factor` (default 1.0).
- Custom key type needs `std::hash` specialization + `operator==` (or supply hash/eq functors). A bad hash → O(n) collisions.
- `reserve(n)` sets bucket count to avoid rehashes.
- No ordering; iteration order is unspecified and unstable across rehash.
- C++23 `std::flat_map` / `flat_set`: sorted `vector` under the hood. O(log n) lookup (binary search) but *contiguous* — much faster iteration and small-map lookups than node-based `map`, at the cost of O(n) insertion. The cache-locality answer when you build once and query many times.

Take-home lesson. `unordered_map` is $O(1)$ on average with a good hash. Adversarial or clustered keys degrade it to $O(n)$ and it never has locality. For small or read-mostly maps, a sorted `vector` / `flat_map` frequently wins on wall-clock despite the "worse" $O(\log n)$.

5. ALGORITHMS (`<ALGORITHM>` , `<NUMERIC>`)

- Know the staples: `sort` , `stable_sort` , `nth_element` , `partial_sort` , `find` / `find_if` , `binary_search` / `lower_bound` / `upper_bound` , `accumulate` , `transform` , `for_each` , `copy` / `move` , `unique` (needs sorted for dedup), `partition` , `rotate` , `min/max_element` , `count_if` .
- `std::sort` = introsort (quicksort → heapsort fallback + insertion sort for small ranges), $O(n \log n)$ worst case, not stable. `stable_sort` is $O(n \log n)$ (or $O(n \log^2 n)$ with less memory), stable.
- `nth_element` = $O(n)$ average selection (quickselect) — the answer to "kth largest without full sort".

```
// kth largest in O(n) average — no full sort needed:
std::nth_element(v.begin(), v.begin() + (k - 1), v.end(),
std::greater{});
int kth = v[k - 1]; // v is now partitioned around the kth largest
// std::greater{} (C++17 CTAD — no <int>) sorts the "largest" side to the
front.
```

- Prefer algorithms + a lambda over hand-rolled loops — clearer intent, often better optimized, and it's what interviewers want to see.
- Ranges (C++20): `std::ranges::sort(v)` (no `.begin()/.end()`), composable lazy views (`views::filter` , `transform` , `take`) with `|` . Views are lazy and non-owning — mind dangling.

```
auto evens_squared = v | std::views::filter([](int x){return x%2==0;})
                        | std::views::transform([](int x){return x*x;});
```

6. MOVE-AWARE & SMALL-OBJECT OPTIMIZATIONS

- SSO (small string optimization): `std::string` stores short strings inline (typically ≤ 15 chars) — no heap alloc. Don't assume `string` always heap-allocates.
 - `std::string_view` / `std::span` — non-owning views over char/element ranges; pass by value (they're cheap), but never outlive the data they point to. Great for read-only params to avoid copies.
-

7. PERF MINDSET (WHAT TO SAY)

1. Measure first — profile, don't guess (perf, VTune, `-fno-omit-frame-pointer`).
 2. Data layout > algorithmic cleverness for small/medium n (cache locality, AoS vs SoA, false sharing).
 3. Avoid needless allocations/copies: `reserve`, `move`, `emplace`, views.
 4. `noexcept` moves so containers move instead of copy.
 5. Prefer contiguous containers; avoid `list` / pointer-chasing.
-

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- Give the access and insert/erase complexity for `vector`, `deque`, `list`, `map`, `unordered_map`.
- What operation invalidates *all* `vector` iterators? What invalidates `unordered_map` iterators?

- Which is contiguous: `vector`, `deque`, `array`? Which is stable under middle insertion: `list`, `vector`?

Level 2 — Apply

1. Pick containers for: LRU cache, task queue with both-end ops, sorted range queries, dedup-by-key. Justify by complexity *and* layout.
2. Fix a loop that erases from a `vector` while iterating (invalidation bug), two ways.
3. Write a `std::hash` + `operator==` for a struct key so it works in `unordered_map`.

Level 3 — Transfer

4. When is `emplace_back` NOT faster than `push_back`? Give a concrete case where it's actively worse.
5. Find the kth largest element without sorting the whole array; state the complexity and the algorithm name.
6. Rewrite a nested filtering-and-transforming loop as a ranges view pipeline; explain what "lazy" buys you and where a dangling view could bite.

Level 4 — Teach

- Explain to a new grad why `std::list` is almost never the right default despite its $O(1)$ insert. If you don't invoke cache lines / pointer-chasing, you're quoting big-O, not teaching.

CONNECTS TO

- File 01 — element lifetimes inside containers; `clear()` vs freeing; placement-new in `vector`'s uninitialized capacity.
- File 02 — `noexcept` moves decide whether `vector` growth moves or copies; `emplace` / `push_back` and move semantics.
- File 03 — iterator categories and tag dispatch are template metaprogramming; `std::sort` needing random-access iterators.
- File 08 — ranges, `span`, `string_view`, `flat_map` are the modern-standard additions layered on these containers.